

Java Coding Interview

1. CUSTOM OBJECTS & RECORDS (Hashing / DTOs)

Explanation:

- When using custom objects as keys in **HashMap**, **HashSet**, or other hashing structures, override `hashCode()` and `equals()`.
- **Java record** provides a concise, immutable data class with built-in `hashCode()`, `equals()`, `toString()`, and getters.
- **Time Complexity:** $O(1)$ avg for HashMap operations assuming proper hash function.

Custom Class Example

```
class Person {
    String name;
    int age;

    Person(String name, int age){ this.name = name; this.age = age; }

    @Override
    public boolean equals(Object o){
        if(this == o) return true;
        if(o == null || getClass() != o.getClass()) return false;
        Person p = (Person) o;
        return age == p.age && Objects.equals(name, p.name);
    }

    @Override
    public int hashCode(){ return Objects.hash(name, age); }
}

// Usage in HashMap
Map<Person, String> map = new HashMap<>();
map.put(new Person("Alice", 30), "Engineer");
System.out.println(map.get(new Person("Alice", 30))); // "Engineer"
```

Java Record Example

```
record PersonRecord(String name, int age) { }

// Usage in HashMap
Map<PersonRecord, String> map = new HashMap<>();
map.put(new PersonRecord("Bob", 25), "Designer");
System.out.println(map.get(new PersonRecord("Bob", 25))); // "Designer"
```

2. CUSTOM ENUMS WITH CONSTRUCTOR

Explanation:

- Enums can have **fields, constructors, and methods**.
- Useful for **constant-specific data**, status codes, or strategy-like behavior.
- Enums are **singleton per constant**, so constructors are called once per enum constant.

Time Complexity: $O(1)$ for enum lookups and usage.

```
enum Status {
    NEW(0, "New order"),
    PROCESSING(1, "Processing order"),
    COMPLETED(2, "Order completed");

    private final int code;
    private final String description;

    Status(int code, String description){
        this.code = code;
        this.description = description;
    }

    public int getCode(){ return code; }
    public String getDescription(){ return description; }
    public boolean isFinal(){ return this == COMPLETED; }
}

// Usage
Status s = Status.NEW;
System.out.println(s.getCode());           // 0
System.out.println(s.getDescription());    // "New order"
System.out.println(s.isFinal());           // false

switch(s){
    case NEW -> System.out.println("Handle new order");
    case PROCESSING -> System.out.println("Handle processing");
    case COMPLETED -> System.out.println("Handle completed");
}
```

3. HASHMAP / LINKEDHASHMAP

Time Complexity: put/get/remove $\rightarrow O(1)$ avg, $O(n)$ worst-case; iteration $\rightarrow O(n)$

```
Map<String,Integer> map = new HashMap<>();
map.put("A",1);           // returns previous value or null
map.put("B",2);           // null
Integer val = map.get("A"); // 1
map.containsKey("B");     // true
map.remove("B");          // 2

LinkedHashMap<String,Integer> insertionOrderMap = new LinkedHashMap<>();
insertionOrderMap.put("A",1); insertionOrderMap.put("B",2); insertionOrderMap.put("C",3);
for(Map.Entry<String,Integer> e : insertionOrderMap.entrySet()){
    System.out.println(e.getKey() + " => " + e.getValue());
}
// Output: A => 1, B => 2, C => 3

LinkedHashMap<String,Integer> accessOrderMap = new LinkedHashMap<>(16, 0.75f, true);
accessOrderMap.put("A",1); accessOrderMap.put("B",2); accessOrderMap.put("C",3);
accessOrderMap.get("A"); // 1, moves "A" to end
accessOrderMap.get("B"); // 2, moves "B" to end
for(Map.Entry<String,Integer> e : accessOrderMap.entrySet()){
    System.out.println(e.getKey() + " => " + e.getValue());
}
// Output: C => 3, A => 1, B => 2
```

4. TREE MAP

Time Complexity: put/get/remove $\rightarrow O(\log n)$, Iteration $\rightarrow O(n)$

```
TreeMap<Integer, String> treeMap = new TreeMap<>();
treeMap.put(3, "C"); treeMap.put(1, "A"); treeMap.put(2, "B");

Integer first = treeMap.firstKey();      // 1
Integer ceiling = treeMap.ceilingKey(2); // 2
Integer floor = treeMap.floorKey(2);     // 2
Integer last = treeMap.lastKey();        // 3
```

5. DEQUE / STACK / QUEUE

Time Complexity: $O(1)$ for push/pop/offer/poll/peek

```
Deque<Integer> dq = new ArrayDeque<>();
dq.offerFirst(1); dq.offerLast(2);      // true
Integer first = dq.peekFirst();          // 1
Integer last = dq.peekLast();            // 2
dq.pollFirst();                          // 1
dq.pollLast();                           // 2

Stack<Integer> stack = new Stack<>();
stack.push(10); stack.push(20);          // 20 on top
Integer top = stack.peek();              // 20
Integer popped = stack.pop();             // 20

Queue<Integer> queue = new LinkedList<>();
queue.offer(1); queue.offer(2);          // true
Integer head = queue.peek();             // 1
Integer removed = queue.poll();           // 1
```

6. PRIORITY QUEUE (MIN/MAX HEAP)

Time Complexity: `offer/poll/remove` $\rightarrow O(\log n)$, `peek` $\rightarrow O(1)$

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
minHeap.offer(5); minHeap.offer(1);           // true
Integer peekMin = minHeap.peek();             // 1
Integer pollMin = minHeap.poll();             // 1

PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());
maxHeap.offer(5); maxHeap.offer(1);           // true
Integer peekMax = maxHeap.peek();             // 5
Integer pollMax = maxHeap.poll();             // 5
```

7. SORTING / COMPARATOR

Time Complexity: $O(n \log n)$

```
// --- List<Integer> ---
List<Integer> nums = Arrays.asList(5, 2, 9, 1);
Collections.sort(nums);                       // Ascending, nums = [1,2,5,9]
nums.sort(Comparator.reverseOrder());         // Descending, nums = [9,5,2,1]

// --- Custom objects: Descending age, ascending name ---
people.sort(
    Comparator.comparing(Person::getAge).reversed()
    .thenComparing(Person::getName)
);
```

8. GENERICS

```
class Box<T> {  
    T val;  
    void set(T v){ val = v; }  
    T get(){ return val; }  
}  
  
<T> void printArray(T[] arr){  
    for(T e: arr) System.out.println(e);  
}
```

9. STREAMS & PARALLEL STREAMS

```
int sum = nums.stream().filter(x -> x%2==0).mapToInt(x->x*x).sum(); // sum of squares  
int psum = nums.parallelStream().filter(x -> x%2==0).mapToInt(x->x*x).sum(); // parallel
```

Tip: Parallel streams help CPU-bound large collections. Avoid for small or I/O-bound data.

10. EXECUTORS

```
ExecutorService ex = Executors.newFixedThreadPool(4);
Future<Integer> res = ex.submit(() -> 42);
Integer value = res.get();           // 42
ex.shutdown();
```

FixedThreadPool Thread Sizing Summary

Task Type	Optimal Threads	Notes
CPU-bound	$\approx$ # of CPU cores	Avoid oversubscription; more threads than cores cause context switching
I/O-bound	$\approx$ # of CPU cores * (1 + wait/compute)	Can exceed cores, threads wait on I/O
Multiple Pools	Adjust per workload	Avoid multiple pools all = cores for CPU-heavy tasks

11. COMPLETABLEFUTURE

```
CompletableFuture<String> f = CompletableFuture.supplyAsync(() -> "Hi");
f.thenApply(r -> r + " World").thenAccept(System.out::println).join(); // prints "Hi World"
```

12. BUFFEREDREADER / SCANNER

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
String line = br.readLine();           // input line as String

Scanner sc = new Scanner(System.in);
int n = sc.nextInt();                  // integer input
String s = sc.nextLine();               // string input
```


13. READ CSV / TEXT FILES

```
try (BufferedReader br = new BufferedReader(new FileReader("data.csv"))) {
    String line;
    while ((line = br.readLine()) != null) {
        String[] values = line.split(",");
        System.out.println(Arrays.toString(values)); // prints array of CSV values
    }
} catch (IOException e) { e.printStackTrace(); }
```

14. COMMON USE CASES

- **HashMap**: Fast key-value lookup
 - **LinkedHashMap**: LRU cache / order-preserving
 - **TreeMap**: Interval queries, ceiling/floor lookups
 - **Deque**: Sliding window, BFS
 - **Stack**: DFS, backtracking
 - **Queue**: BFS, level order
 - **PriorityQueue**: Top K, Dijkstra
 - **Streams / Parallel Streams**: Transformations / CPU parallelism
 - **CompletableFuture**: Async tasks
 - **ExecutorService**: Thread pools
 - **BufferedReader / Scanner**: Fast input
-